Rule of Five/Zero.
STL containers.

титуляр на курса: д-р Тодор Цонков (ttsonkov@gmail.com)

практически примери

теория

От какво ще се състои настоящата лекция?

Динамична памет в класовете. default/delete на специални функции (C++ 11). Пример за клас студент с име (с произволна дължина) и масив от оценки(с произволна дължина). STL контейнери.

Преговор

Деструктор ~T()

Копиращ конструктор T(const T&)

Копиращо присвояване T& operator=(const T&)

Move конструктор T(T&&) (C++11)

Move присвояване T& operator=(T&&) (C++11)

=default и =delete могат да се използват за контрол върху тях.

Пример

Кога да ползваме =delete?

=delete е добра идея винаги, когато трябва изрично да забраним определена операция, защото тя няма смисъл, е опасна или нарушава дизайна на класа.

Пример:

Класът не трябва да се копира (owning ресурс) - най-честият и правилен случай.

Класът притежава уникален ресурс: файл, сокет, mutex, GPU ресурс, OS handle
Копиране би довело до double free / undefined behavior.

```
class File {
    FILE* f_;
public:
    File(const char* name) : f_(fopen(name, "r"))
{}  ~File() { if (f_) fclose(f_); }

    File(const File&) = delete;
    File& operator=(const File&) = delete;

    File(File&&) noexcept = default;
    File& operator=(File&&) noexcept = default;
};
```

Още примери за =delete

Примери

```
class Student {
public:
    Student(int id);
    Student(double) = delete;   //
забраняваме неясни конверсии
};
```

Примери

```
class Student {
    std::string name_;
public:
    Student() = delete;
    explicit
Student(std::string name) :
name_(std::move(name))
{}
};
```

класът няма смисъл без дефолтни данни

Примери за функции

```
void log(int) = delete;
void log(double) = delete;

void log(const std::string& msg) {
    std::cout << msg;
}
```

Пример

Rule of Zero

При използване на класове, които правилно управляват ресурси (RAII) — std::string, std::vector, std::unique_ptr — няма нужда от специални функции: компилатора ги генерира → по-малко грешки, по-чист код.

Дизайн: предпочитаме Rule of Zero
Когато можем: използваме std::string и std::vector<double> за name и grades.
Минимален код, всички специални функции генерирани безопасно.

```
#include <string>
#include <vector>
#include <iostream>

class Student {
    std::string name;        // manages its own memory
    std::vector<int> grades; // manages its own memory

public:
    Student(std::string n, std::vector<int> g)
        : name(std::move(n)), grades(std::move(g)) {}

    double average() const {
        if (grades.empty()) return 0.0;
        int sum = 0;
        for (int g : grades) sum += g;
        return static_cast<double>(sum) / grades.size();
    }

    void print() const {
        std::cout << name << " | avg = " << average() <<
'\n'}
};
```

Rule of Five пример

```
include <cstddef>

class Student {
    char* name_;          // heap-allocated C-string
    double* grades_;      // heap-allocated array of grades
    std::size_t n_grades_;
public:
    Student(const char* name, const double* grades, std::size_t
n); ~Student();

    Student(const Student& other);
    Student& operator=(const Student& other);

    Student(Student&& other) noexcept;
    Student& operator=(Student&& other) noexcept;

    // helpers
    void print() const;
    std::size_t size() const { return n_grades_; }
};

//Student.h
```

Rule of Five constructor and destructor

```cpp
#include <iostream>
#include <algorithm> // std::copy
#include "Student.h"

Student::Student(const char* name, const double* grades, std::size_t
n) name_(nullptr), grades_(nullptr), n_grades_(n)
{
    if (name) {
        std::size_t len = std::strlen(name);
        name_ = new char[len + 1];
        std::strcpy(name_, name);
    }
    if (n > 0) {
        grades_ = new double[n];
        std::copy(grades, grades + n, grades_);
    }
}

Student::~Student() {
    delete[] name_;
    delete[] grades_;
}
//Student.cpp
```

Rule of Five copy constructor

```
Student::Student(const Student& other)
 : name_(nullptr), grades_(nullptr), n_grades_(other.n_grades_)
{
    if (other.name_) {
        std::size_t len = std::strlen(other.name_);
        name_ = new char[len + 1];
        std::strcpy(name_, other.name_);
    }
    if (n_grades_ > 0) {
        grades_ = new double[n_grades_];
        std::copy(other.grades_, other.grades_ + n_grades_,
grades_);
    }
}
```

Rule of Five operator=

```cpp
#include <utility> // std::swap

// swap helper
friend void swap(Student& a, Student& b) noexcept {
    using std::swap;
    swap(a.name_, b.name_);
    swap(a.grades_, b.grades_);
    swap(a.n_grades_, b.n_grades_);
}

// copy-assignment via copy-and-swap (strong exception
Student& Student::operator=(const Student& other) {
    if (this != &other) {
        Student tmp(other); // may throw
        swap(*this, tmp);
    }
    return *this;
}

//Student.cpp
```

Rule of Five - move assignment operator and move constructor

```
    // Move constructor
    Student(Student&& other) noexcept
        : name(other.name),
          grades(other.grades),
          n_grades(other.n_grades)
    {
        other.name = nullptr;
        other.grades = nullptr;
        other.n_grades = 0;
    }

    // Move assignment operator
    Student& operator=(Student&& other) noexcept
{       if (this != &other) {
            // release current resources
            delete[] name;
            delete[] grades;

            // steal resources
            name = other.name;
            grades = other.grades;
            n_grades = other.n_grades;

            // leave other in valid state
            other.name = nullptr;
            other.grades = nullptr;
            other.n_grades = 0;
        }
        return *this;
    }
```

Rule of Zero пример

```
#include <string>
#include <vector>
#include <iostream>

class StudentRAII {
    std::string name_;
    std::vector<double> grades_;
public:
    StudentRAII() = default;
    StudentRAII(std::string name, std::vector<double>
grades) name_(std::move(name)), grades_(std::move(grades))
{}
    void print() const {
        std::cout << "Name: " << name_ << ", grades:";
        for (double g : grades_) std::cout << ' ' << g;
        std::cout << '\n';
    }
};

//Student.cpp
```

STL контейнери

STL контейнерите и помощните класове:

управляват динамична памет
спазват RAII
имат правилни copy/move семантики
освобождават ресурсите автоматично
няма нужда от delete

Пример

Примери за STL типове:

```
std::string
std::vector
std::array
std::map
std::unique_ptr
std::shared_ptr
```

std::array има фиксиран размер (compile-time), а std::vector има динамичен размер (run-time).

std::unique_ptr
std::shared_ptr ще бъдат разгледани в детайли в следващи лекции.

Пример за std::vector

```
struct Student {
    std::string name;
    double grade;
};

int main() {
    std::vector<Student> students;
    students.push_back({"Ivan", 5.50});
    students.push_back({"Maria", 6.00});
    students.push_back({"Georgi", 4.80});

    for (const auto& s : students) {
        std::cout << s.name << " -> " <<
s.grade << '\n';
    }
}
```

Пример за std::array

```
//Знае се броя по време на компилация!
#include <array>
#include <iostream>

int main() {
    std::array<double, 3> position = {1.5,
2.0, -0.5};

    std::cout << "x=" << position[0]
              << " y=" << position[1]
              << " z=" << position[2] <<
'\n';
}
```

std::array

Тип контейнер: Фиксиран масив
Размер: Фиксиран (compile-time)
Памет: Вътре в обекта (няма heap)
Динамично разширяване: Не
Случаен достъп []: Да – O(1)
Вмъкване в края: Не
Вмъкване в средата: Не
Изтриване в средата: Не
Cache-friendly: Да (много)
Допълнителна памет: Не
Copy семантика: Пълно копие
Move семантика: Пълно преместване
Rule of Zero: Да
Кога се използва: Малък, фиксиран размер

std::vector

Тип контейнер: Динамичен масив
Размер: Динамичен (run-time)
Памет: Heap (непрекъсната)
Динамично разширяване: Да
Случаен достъп []: Да – O(1)
Вмъкване в края: Да – amort. O(1)
Вмъкване в средата: O(n)
Изтриване в средата: O(n)
Cache-friendly: Да (много)
Допълнителна памет: Малко (capacity)
Copy семантика: Пълно копие
Move семантика: Евтино
Rule of Zero: Да

Кога се използва: Най-често използваният контейнер

Пример

Пример кога да ползваме std::list

Симулация на плейлист, където:

1)често трябва да се махат и добавят песни в средата

2)пазят се валидни итератори

3)няма нужда от бърз достъп по индекс

✔ Защо list?

1)O(1) insert/erase навсякъде

2)Итераторите не се инвалидират

```
#include <list>

int main() {
    std::list<int> playlist = {1, 2, 3, 4};

    auto it = playlist.begin();
    std::advance(it, 2); // позиция преди 3

    playlist.insert(it, 99);  // добавяме в средата
    playlist.remove(2);       // махаме песен

    for (int song : playlist) {
        std::cout << song << ' ';
    }
}
```

Въпроси?

Благодаря за вниманието!